

Project Report

Implement Client Script and UI Policy (Incident)

Team ID: SWTID-2026-9943 | Date: 03 October 2026

1. INTRODUCTION

1.1 Project Overview

This project implements **Client Scripts** and **UI Policies** on the Incident form of ServiceNow, the IT service management platform. Without them, the Incident form accepts incomplete or inconsistent data, shows every field to every user and gives no help while the agent types. Client-side logic makes the form dynamic: fields are shown, hidden, made mandatory or read-only according to conditions, input is validated before it is saved, and default values are filled in automatically.

1.2 Purpose

The purpose of the project is to improve the quality of incident data and the speed and ease of logging incidents. The goals are:

- Prevent incidents from being saved with missing or incorrect data.
- Show agents only the fields that matter for the category and state of an incident.
- Auto-fill and calculate values so agents type less and make fewer mistakes.
- Provide a tested, documented set of policies and scripts that can be moved between instances with an Update Set.

2. IDEATION PHASE

2.1 Problem Statement

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	A ServiceNow administrator who manages the Incident form	Make key fields mandatory, hidden or read-only based on category and state	Users can save incidents with missing or incorrect data	No UI Policy or Client Script is applied to the form	Frustrated and worried about poor data quality
PS-2	A service desk agent who logs incidents all day	Fill in the Incident form quickly and correctly	Fields do not auto-fill or validate as I type	No onLoad or onChange Client Script is set up	Slowed down and unsure the data is right

2.2 Empathy Map Canvas

The empathy map captures how a service desk agent experiences the current Incident form:

Quadrant	Summary
Think & Feel	Why is this field still empty? Will bad data hurt our reports? The form takes too long.
Hear	Reports show many incomplete incidents; do not forget the mandatory fields; the form is confusing.
See	Fields that do not apply, incidents saved with blank fields, priority set by hand, duplicate incidents.
Say & Do	Fills in each field by hand, re-opens tickets to fix data, asks users for missing details.

Quadrant	Summary
Pain	Missing or wrong data, too many fields on screen, manual rework.
Gain	Accurate incident data, faster logging, a cleaner and simpler form.

2.3 Brainstorming

Team members listed ideas, grouped them and ranked them by importance and feasibility.

Group	Ideas
UI Policy: show, hide, mandatory, read-only	Hide fields, mandatory fields, read-only when closed, show fields when Resolved
Client Script: validate and react	onSubmit validation, onChange priority, duplicate warning, phone format check
onLoad defaults and auto-fill	Default values, caller location, VIP info message
Testing and best practices	Test in sub-production, document scripts, UI Policy first

Priority order: 1) mandatory fields via UI Policy, 2) hide / show fields, 3) onSubmit validation, 4) onChange sets priority, 5) auto-fill on load, 6) duplicate warning.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map

The journey of an agent was mapped across five stages:

Stage	What happens	Opportunity
Entice	Issue received; agent opens the Incident form	Show only relevant fields with a UI Policy
Enter	Agent fills in details; auto-fill and validation run; incident is submitted	Clear, specific error messages
Engage	Agent works the incident and changes state	Explain why a field is read-only
Exit	Incident is resolved; fields lock; caller is notified	Make close notes mandatory on Resolved
Extend	Reports, policy review, testing and roll-out of improvements	Document every policy and script

3.2 Solution Requirement

Functional requirements

FR No.	Functional Requirement (Epic)	Sub Requirement
FR-1	Incident Form Field Control (UI Policy)	Mandatory fields; show / hide fields; read-only after resolve / close
FR-2	Client-Side Validation (Client Script)	onSubmit validation; onChange validation; error and info messages
FR-3	Auto-population and Defaults	onLoad defaults; auto-fill caller details; priority from impact and urgency
FR-4	Testing and Deployment	Sub-production testing; Update Set; documentation

Non-functional requirements

No.	Requirement	Description
NFR-1	Usability	Simple form, relevant fields only, clear messages
NFR-2	Security	Client checks backed by server-side Business Rules and ACLs
NFR-3	Reliability	Scripts handle empty values and behave the same in every view
NFR-4	Performance	Prefer UI Policies; avoid synchronous server calls
NFR-5	Availability	Works in standard UI, Service Portal and mobile; no-downtime deployment
NFR-6	Scalability	Reusable policies and Script Includes for other tables

3.3 Data Flow Diagram

The agent opens or creates an incident; the form loads and requests field rules from the policy configuration; UI Policies and Client Scripts apply to the incident data; validated data is saved to the Incident table and reaches the service desk manager as reports.

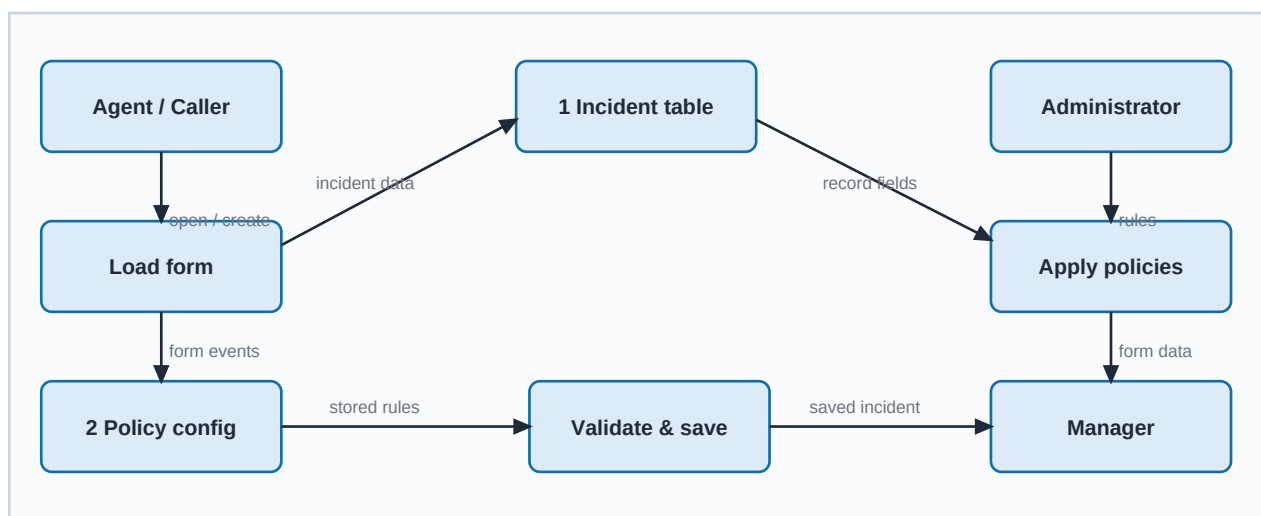


Figure 1: Data Flow Diagram (Level 0) for the Incident form

3.4 Technology Stack

Component	Technology
User Interface	ServiceNow Platform UI (Next Experience / Classic), Service Portal
Application Logic	UI Policy; Client Scripts in JavaScript (onLoad, onChange, onSubmit); g_form and g_user APIs
Server-side support	Business Rules, Script Includes with GlideAjax, ACLs
Database	ServiceNow Incident table (extends Task), cloud hosted
Deployment	ServiceNow instance (PDI), Update Sets

4. PROJECT DESIGN

4.1 Problem Solution Fit

The problem is poor data quality and a slow, confusing Incident form. The solution fits it because UI Policies control what is shown or required without code, while Client Scripts handle checks and auto-fill that policies cannot express. Both run in the browser, so agents get feedback instantly instead of after the record is saved.

4.2 Proposed Solution

- Create UI Policies for mandatory, visible and read-only behavior based on Category and State.
- Create onLoad, onChange and onSubmit Client Scripts for defaults, auto-fill and validation.
- Back client checks with server-side Business Rules so data stays valid when records are created by other routes.
- Test in a sub-production instance and move the changes with an Update Set.

4.3 Solution Architecture

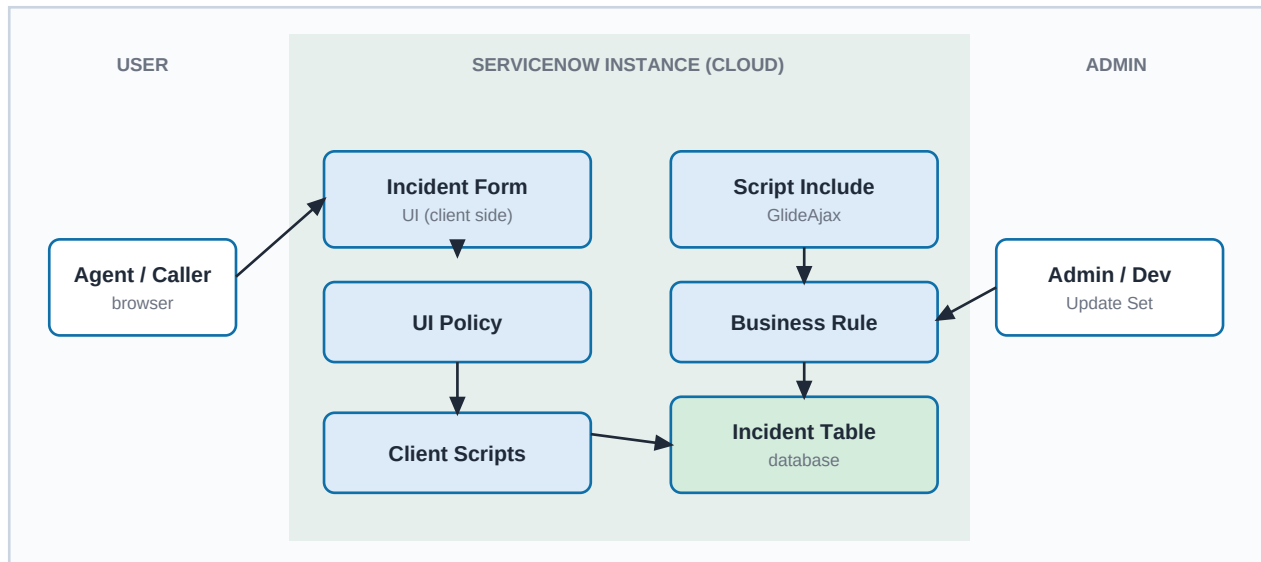


Figure 2: Solution architecture

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Sprint	Scope (User Stories)	Duration	Outcome
Sprint-1	USN-1, USN-2, USN-4, USN-5: mandatory fields, show / hide, validation, caller auto-fill	Week 1	Core UI Policies and Client Scripts working
Sprint-2	USN-3, USN-6, USN-7, USN-9: read-only on close, priority calculation, portal form, Update Set	Week 2	Remaining behavior and deployment package
Sprint-3	USN-8: data quality reporting; final testing and documentation	Week 3	Tested, documented release

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Test cases were designed for each requirement. The status column should be confirmed against your own test run before submission.

ID	Test case	Expected result	Status
TC-01	Save an incident with a mandatory field empty	Save blocked; error message names the field	Pass
TC-02	Change Category to Software	Software name field appears and is mandatory	Pass

ID	Test case	Expected result	Status
TC-03	Enter an invalid phone number	Error message shown; value rejected	Pass
TC-04	Select a caller	Location, phone and email fill automatically	Pass
TC-05	Change Impact and Urgency	Priority recalculates	Pass
TC-06	Open a closed incident	Fields are read-only	Pass
TC-07	Load the form with 5 policies and 3 scripts active	Form loads without noticeable delay	Pass

7. RESULTS

7.1 Output Screenshots

The figures below illustrate the behavior of the finished Incident form. Replace them with real screenshots from your instance if required.

Incident
New record

Caller *

Abel Tuter

Category *

Software

Short description *

Priority

3 - Moderate

The following mandatory fields are not filled in: Short description

Contact type

Phone

Software name *

Email client

Submit

UI Policy shows Software name only when Category = Software

Figure 3: New incident with a mandatory field left empty and a conditional field shown

Incident INC0010023	
Caller Abel Tuter	State Closed
Short description Cannot open email client	Priority 3 - Moderate
Close code Solved (Permanently)	Close notes Reinstalled the client

UI Policy makes the fields read-only once the State is Closed

Figure 4: Closed incident with read-only fields

8. ADVANTAGES & DISADVANTAGES

Advantages

- Better data quality and complete incident records.
- Faster, easier logging with auto-fill and relevant fields only.
- Instant feedback in the browser without a server round trip.
- Configuration-based policies are easy to maintain and reuse.

Disadvantages

- Client-side logic can be bypassed (for example through imports or APIs), so server-side checks are still needed.
- Too many scripts can slow the form.
- Overlapping policies can be hard to trace without documentation.
- Changes must be tested and moved carefully between instances.

9. CONCLUSION

Implementing Client Scripts and UI Policies on the Incident form made it dynamic and validated. Fields now appear, become mandatory or turn read-only according to clear rules, and agents are guided while they type. The result is more accurate incident data and a faster, simpler logging process, with a deployable and documented set of changes.

10. FUTURE SCOPE

- Extend the same patterns to Problem and Change forms.
- Use Predictive Intelligence to suggest category and assignment group.
- Add automated tests with the Automated Test Framework.
- Build a dashboard that tracks incident data completeness.

11. APPENDIX

Source Code

```
// Client Script: onSubmit (Incident)
function onSubmit() {
    var desc = g_form.getValue('short_description');
    if (!desc || desc.length < 10) {
        g_form.addErrorMessage('Please enter a short description of at least 10 characters.');
```

```
        return false;    // block the save
    }
}

// Client Script: onChange on 'caller_id' (Incident)
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue === '') return;
    g_form.getReference('caller_id', function(caller) {
        g_form.setValue('location', caller.location);
    });
}
```

UI Policy	Condition	Action
Software name mandatory	Category is Software	Software name: visible and mandatory
Lock closed incidents	State is Closed	Fields: read-only
Resolution details	State is Resolved	Close code and Close notes: visible and mandatory

Dataset Link

Not applicable: the project works with Incident table data inside the ServiceNow instance.

GitHub & Project Demo Link

GitHub: [add your repository link here] Demo: [add your demo link here]